

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования
«Гродненский государственный университет имени Янки Купалы»

Факультет математики и информатики
Кафедра системного программирования и компьютерной безопасности

КИСЕЛЕВСКИЙ ДАНИИЛ АНДРЕЕВИЧ

**Проектирование, создание и обработка на языке Prolog справочно-
информационной системы «Покупка товаров для гейминга»**

Курсовой проект по учебной дисциплине
«Математические основы интеллектуальных систем»
студента 2 курса специальности
1-40 03 01 «Искусственный интеллект»

Научный руководитель
Просвирнина Ирина Борисовна,
доцент кафедры
системного программирования и
компьютерной безопасности,
кандидат физико-математических
наук, доцент

Гродно 2025

РЕЗЮМЕ

Курсовой проект: 42 с., 8 рис., 4 источника.

ПАРАДИГМА ДЕКЛАРАТИВНОГО ПРОГРАММИРОВАНИЯ, ЯЗЫК PROLOG, ПРОЕКТИРОВАНИЕ И СОЗДАНИЕ СПРАВОЧНО-ИНФОРМАЦИОННОЙ СИСТЕМЫ «ТОВАРЫ ДЛЯ ГЕЙМИНГА» НА ЯЗЫКЕ PROLOG.

Объект исследования: справочно-информационные системы.

Предмет исследования: покупка товаров для гейминга.

Цель работы: реализация справочно-информационной системы «покупка товаров для гейминга» на языке Prolog.

Методы исследования: проектирование, разработка, тестирование, анализ литературы.

Исследования и разработки: изучение истории языка Prolog, анализ парадигмы декларативного программирования, проектирование и создание справочно-информационной системы для поиска и покупки товаров для гейминга, сравнение с альтернативными подходами реализации экспертных систем, анализ возможных улучшений программы.

Область возможного практического применения: область возможного практического применения включает в себя использование системы для практичной каталогизации в сфере магазинов электроники, удобный поиск, сортировку и сравнение товаров в интернет-магазинах, а также для создания экспертной системы в области электронных товаров.

Автор работы подтверждает, что приведённый в курсовом проекте расчётно-аналитический материал правильно и объективно отражает состояние исследуемого процесса, а все заимствованные из литературных и других источников теоретические, методологические и методические положения и концепции сопровождаются ссылками на их авторов.

SUMMARY

Course project: 42 pp, 8 fig., 4 ref.

THE PARADIGM OF DECLARATIVE PROGRAMMING, THE PROLOG LANGUAGE, THE DESIGN AND CREATION OF THE REFERENCE INFORMATION SYSTEM "GAMING PRODUCTS" IN THE PROLOG LANGUAGE.

The object of the research: reference information systems.

The subject of the research: purchase of gaming products.

The purpose of the research: the implementation of the reference information system "purchase of goods for gaming" in the Prolog language.

The research methods: design, development, testing, literature analysis.

Research and development: study of the history of the Prolog language, analysis of the declarative programming paradigm, design and creation of a reference information system for searching and buying gaming products, comparison with alternative approaches to implementing expert systems, analysis of possible program improvements.

The area of possible practical application: the scope of possible practical application includes using the system for practical cataloging in the field of electronics stores, convenient search, sorting and comparison of goods in online stores, as well as for creating an expert system in the field of electronic goods.

The author of work confirms that the design and analytical materials of course project reflect the state of the process under research correctly and impartially, and all theoretical, methodological and methodical statements and conceptions borrowed from the literature and other sources are accompanied by references to their authors.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. ИЗУЧЕНИЕ ЛОГИЧЕСКОЙ (ДЕКЛАРАТИВНОЙ) ПАРАДИГМЫ ПРОГРАММИРОВАНИЯ НА ПРИМЕРЕ ЯЗЫКА ЛОГИЧЕСКОГО ПРОГРАММИРОВАНИЯ PROLOG	6
1. Основные парадигмы программирования	6
Императивная парадигма	6
Декларативная парадигма	6
2. Основы логической парадигмы программирования на примере языка Prolog	7
Язык программирования Prolog	7
ГЛАВА 2. МАТЕМАТИЧЕСКОЕ ОБОСНОВАНИЕ СТРУКТУРЫ И СИНТАКСИСА ЯЗЫКА PROLOG С ТОЧКИ ЗРЕНИЯ ИСЧИСЛЕНИЯ ПРЕДИКАТОВ	9
1. Исчисление предикатов	9
2. Синтаксис языка Prolog	9
3. Выводы	10
ГЛАВА 3. ПРОЕКТИРОВАНИЕ И СОЗДАНИЕ СПРАВОЧНО- ИНФОРМАЦИОННОЙ СИСТЕМЫ «ТОВАРЫ ДЛЯ ГЕЙМИНГА»	11
1. Проектирование	11
Проектирование фактов	11
Проектирование правил	12
Проектирование меню программы	13
2. Создание справочно-информационной системы	13
Создание базы знаний	14
Создание правил	14
Создание меню программы	22
3. Выводы	24
ГЛАВА 4. ТЕСТИРОВАНИЕ И ТРАССИРОВКА ПРОГРАММЫ	25
1. Анализ работоспособности программы	25
2. Анализ с использованием трассировки	29
3. Выводы	32
ГЛАВА 5. ПОСТРОЕНИЕ ДЕРЕВЬЕВ ЦЕЛИ С ИСПОЛЬЗОВАНИЕМ ЯЗЫКА UML	34
1. Построение UML-дерева	34
2. Выводы	36
ЗАКЛЮЧЕНИЕ	37
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	39
ПРИЛОЖЕНИЕ А	40

ВВЕДЕНИЕ

В настоящее время остро встает тема экспертных систем. Ныне они набирают особую популярность в контексте нейронных сетей и больших языковых моделей. Но не только в языковых моделях находится их потенциал. Экспертные системы (или справочно-информационные) используются в медицине, финансовом секторе, а также в сфере интернет магазинов при проектировании каталогов товаров. С этой задачей может справиться логический язык программирования Prolog.

Язык Prolog основан на математической логике, а конкретно на логике предикатов первого порядка. Язык является декларативным, то есть состоит из фактов и правил.

На данный момент не на всех сайтах магазинов присутствуют удобные, специфичные и полезные фильтры, такие как сравнение двух товаров, или динамичная фильтрация. И в рамках этой курсовой будет разработана часть экспертной системы, способная к сортировке, сравнению и поиску по фильтрам товаров для гейминга.

В данной курсовой будет представлена справочно-информационная система, способная помочь выбрать необходимые товары для гейминга. Цель работы — спроектировать, разработать и протестировать экспертную систему, способную помочь пользователям выбрать оптимальные геймерские компоненты, комплектующие и периферийные устройства.

Для успешного выполнения работы необходимо решить следующие задачи:

1. Изучить теоретические основы логического программирования на языке Prolog.
2. Проанализировать область товаров для гейминга, выбрать основные и наиболее релевантные категории товаров, выбрать наиболее важные характеристики каждого из товаров.
3. Спроектировать и реализовать базу знаний, состоящую из фактов.
4. Создать несколько предикатов, способствующих помочь пользователю в поиске необходимых товаров для гейминга.
5. Разработать удобный пользовательский интерфейс.
6. Провести тестирование с использованием трассировки программы.

В данной курсовой будет представлена справочно-информационная система, способная помочь пользователю в выборе необходимых товаров для гейминга.

Проект состоит из введения, пяти глав, заключения, списка использованных источников и приложения.

ГЛАВА 1. ИЗУЧЕНИЕ ЛОГИЧЕСКОЙ (ДЕКЛАРАТИВНОЙ) ПАРАДИГМЫ ПРОГРАММИРОВАНИЯ НА ПРИМЕРЕ ЯЗЫКА ЛОГИЧЕСКОГО ПРОГРАММИРОВАНИЯ PROLOG

1. Основные парадигмы программирования

Каждый современный язык программирования руководствуется определенными правилами, стилем и принципами написания кода. Языки, имеющие схожие идеи и концепции, объединяются в группы. Исходя из этих групп существуют разные подходы к программированию — парадигмы программирования.

Императивная парадигма

Самой распространенной парадигмой является императивная (англ. imperative — приказ, повеление). Из названия можно заметить, что императивные языки представляют собой набор инструкций (команд), а сама программа выполняется последовательно. Также к основным чертам императивного программирования относятся:

1. Использование переменных.
2. Использование оператора «присваивание».
3. Использование составных выражений.
4. Использование подпрограмм и др.

Самый яркий и известный пример императивного программирования — объектно-ориентированное программирование. В нем написание программ осуществляется путем создания классов и объектов. Классы, в свою очередь, включают поля (переменные) и методы (функции), а также имеют возможности к абстракции, инкапсуляции, наследованию и полиморфизму.

Декларативная парадигма

Главное отличие декларативного программирования от императивного в способе решения задачи. В императивном программировании задается способ получения решения задачи с помощью команд. В декларативном — описывается ожидаемый результат, задается спецификация решения задачи.

В программах, написанных с использованием декларативных языков, нет переменных или операторов присваивания. Вместо этого, в них используются исполнимые спецификации (декларации).

2. Основы логической парадигмы программирования на примере языка Prolog

Одним из подвидов декларативного программирования является логическое программирование, которое хорошо реализовано в языке Prolog.

Главная особенность логического программирования — логическое следование. Используя за основу математическую логику, программа на языке Prolog способна находить и выводить логическую последовательность используя понятия фактов и правил, а также логику предикатов.

Язык Prolog также широко ценится благодаря удобной и эффективной способности к обработке списков.

Логическое программирование широко применяется в следующих сферах:

1. В искусственном интеллекте и экспертных системах логические языки отлично подходят для представления фактов и провал, а также для автоматического получения новых знаний из уже существующих.
2. Обработка естественного языка (NLP). Логическое программирование помогает в синтаксическом и семантическом анализе.
3. Автоматическое доказательство теорем.
4. Планирование и решение задач с ограничениями. В сферу входят составление оптимального расписания занятий, графика работы, производственных планов, распределение ограниченных ресурсов и др.

Язык программирования Prolog

Язык Prolog (Programming in Logic) наиболее яркий и известный представитель языка, основанного на принципах логического программирования. Он был разработан в 1972 году Аленом Колмерозером и использовал за основу математическую логику предикатов, а точнее математическую логику предикатов первого порядка, а также дизъюнкторы Хорна. Можно сказать, что Prolog основан на принципах математической логики, способных описать объекты и их взаимоотношения.

Основными концепциями в прологе являются факты, правила и запросы.

Факты — безусловные утверждения, описывающие свойства и отношения объектов. Факты имеют вид `имя_предиката(аргумент1, аргумент2, ...)`, где `имя_предиката` определяет отношение, а аргументы — объекты, участвующие в этом отношении.

Правила — это условные зависимости. Они позволяют выводить новые факты на основе уже существующих. Правило состоит из заключений и

посылок, соединенных оператором $:-$ (читается как «если»). Пример: $\text{сын}(X, Y) :- \text{родитель}(Y, X)$.

Запросы — это метод взаимодействия с программой. Иными словами, запросом является вопрос к базе знаний, на который программа пытается найти и вывести ответ, используя механизмы унификации и бэктрекинга для перебора возможных решений.

Также стоит упомянуть про термы. Термами в прологе являются числа, атомы (символьные константы) и переменные (в отличие от атомов, они начинаются с заглавной буквы). Более сложные структуры данных создаются с помощью составных термов (структур).

Как было упомянуто ранее, в прологе реализована работа со списками. Список — последовательность элементов, содержащая термы любого вида. Списки записываются в квадратных скобках и обрабатываются рекурсивно с разделением на голову (первый элемент) и хвост (все остальные элементы).

3. Выводы

Логическое программирование является подвидом декларативного. Наиболее яркое воплощение логического программирования реализовано в языке Prolog.

Пролог основан на математической логике и логическом следовании, концептуально основан на актах, правилах и запросах к базе знаний, структурно состоит из термов и списков.

Язык логического программирования Prolog силен во многих сферах, в том числе в искусственном интеллекте, обработке естественного языка, автоматическом доказательстве теорем и др.

ГЛАВА 2. МАТЕМАТИЧЕСКОЕ ОБОСНОВАНИЕ СТРУКТУРЫ И СИНТАКСИСА ЯЗЫКА PROLOG С ТОЧКИ ЗРЕНИЯ ИСЧИСЛЕНИЯ ПРЕДИКАТОВ

1. Исчисление предикатов

Исчисление предикатов — это формальное исчисление высказываний с переменными. Иными словами, исчисление предикатов подразумевает добавление переменных в высказывания (пример: высказывание — «маша любит кашу», предикат — «X любит кашу»). Таким образом, исчисление предикатов — это, своего рода, расширение логики высказываний.

В исчислении предикатов, помимо самих предикатов, используются иные объекты, такие как переменные (x, y, z, ...), константы (a, b, c, ...) функции (f(x), g(x), ...), предикаты (P(x1, x2, ..., xn), Q(y1, y2, ..., yn)), логические связки (конъюнкция, дизъюнкция, импликация, отрицание, ...) и кванторы (всеобщности и существования).

Все эти элементы в совокупности позволяют в полной мере использовать исчисление предикатов для построения выражений и формирования связей.

Таким образом можно сказать, что исчисление предикатов — это мощный инструмент математической логики, способный строить логические связи и, используя правила вывода, получать новые данные.

2. Синтаксис языка Prolog

Как было сказано ранее, Prolog — язык, основанный на парадигме декларативного программирования, а точнее логического программирования. Основой пролога выступает исчисление предикатов первого порядка.

Базой языка являются факты, правила и логический вывод. Ниже приведен пример факта, правила и вывода.

```
человек(Сократ) . % факт
смертен(X) :- человек(X) . % правило

?- смертен(Сократ) . % вывод (запрос в базу знаний)
```

Листинг 2.1 Пример факта, правила и вывода в Prolog

Примечание Источник: собственная разработка

На вопрос «смертен ли Сократ?» программа выдаст ответ «true.». Действительно, если Сократ — человек, и X смертен если X — человек, значит Сократ — смертен. Этот пример хорошо показывает особенности и отличия декларативного языка от императивного. Однако, при кажущейся простоте языка, он способен создавать сложные, большие и многофункциональные логические конструкции.

3. Выводы

Язык программирования Prolog основан на исчислении предикатов первого порядка. В своем арсенале он имеет логический вывод, который похож на метод дедукции. Это позволяет языку, на основе декларативно заданных фактов и правил, автоматически выводить новые утверждения и отвечать на вопросы.

Приведенный пример наглядно показывает, как синтаксис Prolog позволяет формулировать логические связи, аналогичные тем, что используются в исчислении предикатов.

ГЛАВА 3. ПРОЕКТИРОВАНИЕ И СОЗДАНИЕ СПРАВОЧНО-ИНФОРМАЦИОННОЙ СИСТЕМЫ «ТОВАРЫ ДЛЯ ГЕЙМИНГА»

1. Проектирование

До создания полноценной вопрос-ответной системы, необходимо определить все её объекты и характеристики. Нужно проанализировать задачу, собрать необходимые данные и на основе данных создать факты. После этого создать правила — основные функции справочно-информационной системы. В конце стоит задуматься над пользовательским интерфейсом и удобством использования программы.

Для дальнейшего удобства при создании программы стоит разделить программу на 3 файла: файл facts.pl для хранения всех фактов, файл rules.pl для хранения всех правил и файл program.pl для вывода меню программы.

Программа создается под SWI-Prolog.

Проектирование фактов

Каждая справочно-информационная система (экспертная система) имеет заданный набор данных, или фактов. Перед добавлением этих данных в базу знаний их необходимо выбрать и, возможно, преобразовать, отредактировать.

Вопрос-ответная система «товары для гейминга» должна хранить данные, соответствующие игровой индустрии. Товары удобно разделить на категории, к примеру клавиатуры, игровые мыши, гарнитуры и др. В этом курсовом проекте было выбрано 8 категорий (мыши, мониторы, процессоры, накопители, клавиатуры, видеокарты, оперативная память, гарнитуры), в каждой категории по 7 товаров.

Обращаясь к системе, пользователю необходимо сделать выбор между товарами на основании имеющихся характеристик об этих товарах. У каждой категории товаров есть множество уникальных характеристик и стоит выбрать только самые значимые. Однако для каждой категории товаров существуют смежные характеристики, такие, что присутствуют во всех категориях сразу. Следующие характеристики были выбраны как общие для всех: категория, бренд, модель, цена.

Ниже приведен фрагмент данных, записанных в таблицу Excel для удобства хранения.

Мониторы

Category	Brand	Model	Price	Diagonal Inch	Resolution	Refresh Rate HZ	Panel Type	Adaptive Sync	Aspect Ratio	HRD Support
monitor	Xiaomi	Gaming Monitor G27i	523	27	1920x1080	165	IPS	AMD FreeSync Premium	16:9	HDR10
monitor	LG	Ultra Gear 24GS60F-B	556	23.8	1920x1080	180	IPS	AMD FreeSync	16:9	HDR10
monitor	Asus	Rog Strix XG27ACS	1199	27	2560x1440	180	IPS	AMD FreeSync	16:9	HDR10
monitor	Samsung	Odyssey G4	849	25	1920x1080	240	IPS	AMD FreeSync Premium	16:9	HDR10
monitor	Asus	TUF Gaming	649	27	1920x1080	180	IPS	AMD FreeSync Premium	16:9	-
monitor	MSI	MAG 256F	546	24,5	1920x1080	180	IPS	VESA Adaptive-Sync	16:9	-
monitor	Philips	Evnia	799	27	2560x1440	180	VA	AMD FreeSync, VESA Adaptive-Sync	16:9	HDR10

Процессоры (CPU)

Category	Brand	Model	Price	Cores	Threads	Base Clock GHZ	Boost Clock gHZ	Socket	TDP W	Integrated Graphics
cpu	AMD	Ryzen 5 7500F	480	6	12	3,7	5	AM5	65	-
cpu	AMD	Risen 7 7800X3D	1405	8	16	4.2	5	AM5	120	AMD Radeon Graphics
cpu	Intel	Core i5-13600KF	618	14	20	3,5	5,1	LGA1700	125	-
cpu	Intel	Core i3-12100F	168	4	8	3,3	4,3	LGA1700	58	-
cpu	Intel	Core i7-14700KF	1140	20	28	3,4	5,6	LGA1700	125	-
cpu	AMD	Ryzen 9 5900X	934	12	24	3,7	4,8	AM4	105	-
cpu	Intel	Core i9-14900K	1449	24	32	3,2	6	LGA1700	125	Intel UHD Graphics 770

Накопители (SSD/HDD)

Category	Brand	Model	Price	Storage Type	Capacity GB	Interface	Form Factor Inch	Read Speed MBpS	Write Speed MBpS	RPM
storage	Toshiba	L200	206	HDD	1000	SATA 3.0	2.5	135	130	5400
storage	HP	1TB	232	HDD	1000	SATA 3.0	2.5	190	180	7200
storage	HP	600GB	641	HDD	600	SATA 2.0	2.5	230	220	10000
storage	Apacer	AS228P4	106	SSD	512	PCI Express x4 3.0	M.2	2100	1500	-
storage	Apacer	AS350X	180	SSD	1000	SATA 3.0	2.5	560	540	-
storage	Kingston	NV3	199	SSD	1000	PCI Express x4 4.0	M.2	6000	4000	-
storage	Kingston	A400	113	SSD	480	SATA 3.0	2.5	500	450	-

Рисунок 3.1 – Фрагмент таблицы с данными

Примечание Источник: собственная разработка

В результате данные о 56 товарах и их характеристиках удобно хранятся в таблице, что упрощает дальнейший перенос этих данных в базу знаний программы.

Проектирование правил

Задача справочно-информационной системы — помочь пользователю с выбором товаров. Для этого существует множество методов, направленных на упрощение представления данных. Это такие методы, как сравнение, сортировка товаров, фильтрация и другие. В рамках этого курсового проекта были выбраны следующие правила: сортировка товаров по выбранной характеристике, сравнение двух товаров одной категории, фильтрация и вывод всех характеристик одного товара.

Правило для вывода отсортированных товаров по выбранной характеристике должно выводить список всех доступных товаров конкретной категории. Эти товары должны быть отсортированы (в рамках данной

курсовой — только по возрастанию) по заранее определенному критерию (характеристике). Если выбранная характеристика товара включает в себя только буквы — сортировка производится по алфавиту. Таким образом, предикат будет иметь два аргумента: категорию и характеристику.

Правило, ответственное за сравнение двух товаров выбранной категории, будет выводить таблицу с тремя столбцами: характеристика, товар1, товар2. Это правило позволит наглядно сравнить два товара и выбрать наиболее подходящий.

Суть фильтрации состоит в том, чтобы отсеять «лишние» товары, не удовлетворяющие по каким-либо характеристикам. Предикат должен будет принимать на вход категорию товаров и список «отфильтрованных» характеристик. Пример списка: [цена $\leq$ 200, бренд $\neq$ «Razer»] (знак $\leq$ означает «меньше, либо равно», а знак $\neq$ означает «не равно», или «не эквивалентно»). С использованием списков, можно выбирать любую существующую характеристику и «фильтровать» товары используя математические знаки, такие как $=$, $\neq$, $>$, $<$ и другие.

Правило для вывода всех характеристик одного товара будет добавлено в целях повышения удобства использования программы. Предикат будет принимать два значения — бренд и модель товара.

Эти правила дают фундаментальные, но широкие возможности для пользователя. Они способны помочь в выборе товаров путем сравнения, сортировки и фильтрации. Главная особенность — правило для фильтрации. Оно позволяет намного шире и масштабнее произвести фильтрацию товаров, чем некоторые типичные реализации фильтров в интернет магазинах и онлайн-сервисах.

Проектирование меню программы

Главная задача при создании меню — сделать его удобным и понятным для пользователя. Так как программа будет выполнена в консольном варианте, то пользователю будет необходимо получать инструкции поэтапно, шаг за шагом. Также стоит отметить, что после исполнения любого правила программа должна будет перейти в начальное состояние, иными словами — вызов меню программы должен быть рекурсивным.

2. Создание справочно-информационной системы

Как было оговорено ранее, программа будет состоять из трех файлов: facts.pl, rules.pl и program.pl. Эти файлы необходимо будет связать при

помощи строки `:- consult('имя_файла.pl')`, которая добавляет все скомпилированные предикаты.

Создание базы знаний

Для создания базы знаний системы необходимо создать предикаты, имеющие в качестве аргументов характеристики товаров. Таким образом, мы получим 56 строк, описывающие 56 товаров.

```
% keyboard(ProductID, Brand, Model, Price, Keyboard_Type,
Switch_Info, Layout_Type, Backlight, Has_Numpad).
keyboard(keyboard1, 'Redragon', 'Nagato RU', 79.9, 'Membrane',
'Membrane', 'Full', 'RGB Rainbow', yes).
keyboard(keyboard2, 'Redragon', 'Fizz RU', 89, 'Mechanical',
'Outemu Brown', 'Compact 60', 'RGB', no).
```

Листинг 3.1 Фрагмент кода из файла facts.pl

Примечание Источник: собственная разработка

Таким образом получается компактная запись всех фактов, которые в дальнейшем будут использоваться в файле с правилами для создания логического вывода.

Создание правил

Для реализации функции сортировки товаров по выбранной характеристике был разработан предикат `find_sorted_by_attribute/2`. Этот предикат принимает на вход два аргумента: `Category` (атом, обозначающий категорию товара, например `gpu`) и `Attribute` (атом, отвечающий за характеристику, по которой будет производится сортировка, например `price`). Задача правила — собрать все товары указанной категории, извлечь значения указанного атрибута с каждого товара и предоставить пользователю отсортированный список.

```
% Rule for sorting
find_sorted_by_attribute(Category, Attribute) :-
    ( \+ current_predicate(Category/_) ->
        format('Error: Unknown category "~w".~n', [Category]),
        fail % Error
    ; \+ attribute_index(Category, Attribute, _) ->
        format('Error: Unknown attribute "~w" for category "~w"
or not defined for sorting.~n', [Attribute, Category]),
```

```

        fail % Indicate failure
    ;
    attribute_index(Category, Attribute, Index),
    current_predicate(Category/Arity), % Get the arity
(number of arguments)
    length(ArgsList, Arity),
    GoalTemplate =.. [Category | ArgsList],
    findall(
        SortValue - item(ID, Brand, Model, Price),
        (
            call(GoalTemplate),
            arg(1, GoalTemplate, ID),
            arg(2, GoalTemplate, Brand),
            arg(3, GoalTemplate, Model),
            arg(4, GoalTemplate, Price),
            arg(Index, GoalTemplate, SortValue)
        ),
        ListOfPairs
    ),
    keysort(ListOfPairs, SortedPairs),
    ( SortedPairs == [] ->
        format('No ~w found matching criteria.~n',
[Category])
    ;
        format('~w sorted by "~w"~n', [Category,
Attribute]),
        print_sorted_list(SortedPairs, Attribute)
    )
).

```

Листинг 3.2 Правило для вывода отсортированных товаров

Примечание Источник: собственная разработка

Работа предиката строится следующим образом:

1. Валидация исходных данных (проверка на существование категории и атрибута). Вспомогательное правило `attribute_index/3` сопоставляет атомарное имя атрибута с его порядковым номером в структуре факта.
2. Формируется список `ListOfPairs` при помощи встроенного предиката `findall/3`. Каждый элемент этого списка представляет пару `SortValue - item(ID, Brand, Model, Price)`, где `SortValue` — значение атрибута, по которому производится сортировка, извлеченное из соответствующего факта `GoalTemplate` с помощью `arg(Index, GoalTemplate, SortValue)`. `item(ID, Brand, Model, Price)` содержит основную идентифицирующую информацию о товаре.

3. Полученный список пар ListOfPairs сортируется с помощью стандартного предиката keysort/2, который упорядочивает элементы по SortValue.

4. Полученный отсортированный список выводится в консоль при помощи правила print_sorted_list/2, который форматирует вывод и отображает необходимые атрибуты для каждого товара категории.

Для предоставления пользователю возможности детально сопоставить характеристики двух товаров одной категории, было разработано правило compare_products/5. Он принимает пять аргументов: Category (категория товаров), Brand1 и Model1 (бренд и модель первого товара), Brand2 и Model2 (бренд и модель второго товара). Задача предиката — найти оба товара в базе знаний и вывести их характеристики в удобном для сравнения формате.

```
compare_products(Category, Brand1, Model1, Brand2, Model2) :-
    ( current_predicate(Category/Arity) -> true ;
    format('Error: Unknown category "~w".~n', [Category]), fail ),
    length(Argsl, Arity), Term1 =.. [Category | Argsl], arg(2,
Term1, Brand1), arg(3, Term1, Model1),
    length(Argsl, Arity), Term2 =.. [Category | Argsl], arg(2,
Term2, Brand2), arg(3, Term2, Model2),
    ( once(Term1) -> true ; format('Error: Product not found:
~w ~w ~w~n', [Category, Brand1, Model1]), fail ),
    ( once(Term2) -> true ; format('Error: Product not found:
~w ~w ~w~n', [Category, Brand2, Model2]), fail ),
    ( arg(1, Term1, ID1), arg(1, Term2, ID2), ID1 == ID2 ->
        format('--- Comparing the same product instance: ~w (~w
~w) ---~n', [ID1, Brand1, Model1]),
        compare_attributes_loop(1, Arity, Category, Term1,
Term2)
    ;
        format('~w: ~w ~w <vs> ~w ~w~n', [Category, Brand1,
Model1, Brand2, Model2]),
        compare_attributes_loop(2, Arity, Category, Term1,
Term2)
    ).

compare_attributes_loop(N, Arity, _, _, _) :- N > Arity, nl.
compare_attributes_loop(N, Arity, Category, Term1, Term2) :-
    N <= Arity,
    arg(N, Term1, Value1), arg(N, Term2, Value2),
    ( attribute_index(Category, AttrNameAtom, N) -> AttrName =
AttrNameAtom ; atom_concat('Attribute ', N, AttrName) ),
    ( format('~w: ~w <vs> ~w~n', [AttrName, Value1, Value2]) ),
    N1 is N + 1,
    compare_attributes_loop(N1, Arity, Category, Term1, Term2).
```

Листинг 3.3 Правило для сравнения двух товаров

Примечание Источник: собственная разработка

Предикат `compare_products/5` начинается с проверки существования указанной категории `Category` с помощью встроенного предиката `current_predicate(Category/Arity)`. Затем динамически формируются шаблоны `Term1` и `Term2` для поиска фактов обоих товаров основе их категории, бренда и модели с использованием `length/2` и `Term =.. [Category | ArgsList]`, а также `arg/3` для постановки бренда и модели в нужные позиции. Предикат `once/1` используется для поиска первого совпадения для каждого товара, а если товар не найден, выводится сообщение об ошибке и завершение работы правила.

Отдельно обрабатываются случаи, если идентификатор найденных товаров совпадают (`ID1 == ID2`). Это значит, что пользователь пытается сравнить товар сам с собой. В этом случае выводится предупреждение о уравнивании двух одинаковых товаров и таблица с товарами, начиная с 1-го атрибута (`ID`). В противном случае, выводится список атрибутов, начиная со 2-го (`Brand`). Сам процесс вывода атрибутов реализован рекурсивно с помощью предиката `compare_attributes_loop/5`.

Предикат `compare_attributes_loop/5` принимает текущий номер атрибута `N`, общую арность `Arity`, категорию и термы двух сравниваемых продуктов. В базовом случае рекурсии, когда `N > Arity`, вывод завершается. При `N < Arity` из обоих термов извлекаются значения `Value1` и `Value2` для атрибута с номером `N` с помощью `arg(N, term, Value)`. Имя атрибута `AttrName` получается из `attribute_index/3`. Затем значения атрибутов для обоих товаров выводятся в формате «ИмяАтрибута: Значение1 <vs> Значение2». После этого `N` увеличивается, и предикат рекурсивно вызывается для следующего атрибута.

Наиболее мощной функцией системы является универсальная фильтрация товаров, реализованная предикатов `find_products/2`. Данный предикат позволяет пользователю находить товары указанной категории `Category`, которые удовлетворяют списку критериев `CriteriaList`. Каждый критерий в списке представляет собой терм вида «Атрибут Оператор Значение» (например, `price <= 500`, `brand == 'AMD'`), где Оператор может быть одним из символов: `==`, `\==`, `<`, `>`, `<=`, `>=`. Предикат находит все товары, соответствующие всем указанным критериям.

```
find_products(Category, CriteriaList) :-
    ( current_predicate(Category/Arity) ->
        true
    ;
        format('Error: Unknown category "%w".~n', [Category]),
```

```

        fail
    ),
    length(ArgsList, Arity),
    GoalTemplate =.. [Category | ArgsList],
    findall(
        item(ID, Brand, Model, Price), % Collect basic info
        (
            call(GoalTemplate), % Find a product fact, binding
ArgsList
            check_criteria(CriteriaList, GoalTemplate,
Category),
            arg(1, GoalTemplate, ID),
            arg(2, GoalTemplate, Brand),
            arg(3, GoalTemplate, Model),
            arg(4, GoalTemplate, Price)
        ),
        ResultList
    ),
    print_filter_results(ResultList, Category). % Reuse the
existing print helper

check_criteria([], _GoalTerm, _Category). % Base case: No
criteria left, success!
check_criteria([Criterion|RestCriteria], GoalTerm, Category) :-
    Criterion =.. [Operator, Attribute, Value], % Assumes
binary operator structure
    ( is_comparison_operator(Operator) -> % Check if operator
is valid comparison
        true
    ;
        format('Error: Invalid operator "~w" in criterion "~w".
Use ==, \\==, <, >, =<, >=.\n', [Operator, Criterion]),
        fail
    ),

    ( attribute_index(Category, Attribute, Index) ->
        arg(Index, GoalTerm, FactValue),
        ( (Attribute == total_capacity_gb, Category == ram) ->
            handle_ram_total_capacity(GoalTerm, TotalCapacity),
            ComparisonValue = TotalCapacity
        ;
            ComparisonValue = FactValue
        ),
        ComparisonGoal =.. [Operator, ComparisonValue, Value],
        call(ComparisonGoal), % Fails if the criterion is not
met
        check_criteria(RestCriteria, GoalTerm, Category)
    )

```

```

;
    format('Error: Unknown attribute "~w" for category "~w"
in criteria.~n', [Attribute, Category]),
    fail
).

```

Листинг 3.4 Правило для универсальной фильтрации товаров

Примечание Источник: собственная разработка

Работа предиката начинается с валидации категории `Category`. Затем используется `length/2` и `GoalTemplate =.. [Category | ArgsList]`, подготавливается шаблон `GoalTemplate` для перебора всех фактов данной категории. Основной поиск осуществляется с помощью `findall/3`, который собирает информацию (`item(ID, Brand, Model, Price)`) о тех товарах, для которых предикат `GoalTemplate` успешен, и которые одновременно удовлетворяют всем условиям из `CriteriaList`. Проверка соответствия критериям делегируется вспомогательному предикату `check_criteria/3`.

`check_criteria/3` рекурсивно обрабатывает список критериев. Базовый случай — пустой список критериев, что означает успех и завершение программы. В рекурсивном шаге `Criterion` из списка разбивается на `Operator`, `Attribute` и `Value` с помощью `Criterion =.. [Operator, Attribute, Value]`. Проверяется допустимость оператора через `is_comparison_operator/1`. Затем, с использованием предиката `attribute_index/3`, определяется позиция `Index` указанного `Attribute` в факте. Значение этого атрибута, `FactValue`, извлекается из текущего проверяемого товара `GoalTerm` с помощью `arg(Index, GoalTerm, FactValue)`. Особое внимание уделяется вычисляемым атрибутам, таким как `total_capacity_gb` для оперативной памяти, где значение получается предикатом `handle_ram_total_capacity/2` путем умножения количества плашек оперативной памяти на объем памяти одного модуля.

Ключевым моментом в `check_criteria/3` является динамическое формирование цели сравнения: `ComparisonGoal =.. [Operator, ComparisonValue, Value]`, где `ComparisonValue` — это вычисленное значение атрибута товара. Эта цель затем выполняется с помощью `call(ComparisonValue)`. Если хотя бы один критерий не выполняется, текущая ветка проверки завершается ошибкой. Список найденных товаров `ResultList` выводится с помощью `print_filter_results/2`.

Следующий предикат поможет получить полную информацию о конкретном товаре по его бренду и модели, `find_product_details/2`. Он принимает аргументы `BrandToFind` и `ModelToFind` и осуществляет поиск по всем известным категориям товаров, определенным в `known_categories/1`.

Цель — найти товар и вывести все его характеристики, кроме идентификатора.

```
find_product_details(BrandToFind, ModelToFind) :-
    known_categories([gpu, headset, keyboard, monitor, mouse,
storage, ram, cpu]),
    known_categories(Categories),
    find_product_in_categories(Categories, BrandToFind,
ModelToFind, Found),
    ( Found == true ->
        true
    ;
        format('Product not found: ~w ~w in any known
category.~n', [BrandToFind, ModelToFind])
    ).

find_product_in_categories([], _BrandToFind, _ModelToFind,
false).
find_product_in_categories([Category|RestCategories],
BrandToFind, ModelToFind, Found) :-
    ( try_find_and_print_details(Category, BrandToFind,
ModelToFind) ->
        Found = true
    ;
        find_product_in_categories(RestCategories, BrandToFind,
ModelToFind, Found)
    ).

try_find_and_print_details(Category, BrandToFind,
ModelToFind) :-
    current_predicate(Category/Arity), % Get arity of the
category predicate
    length(ArgsList, Arity),
    GoalTemplate =.. [Category | ArgsList],

    arg(2, GoalTemplate, BrandToFind),
    arg(3, GoalTemplate, ModelToFind),

    call(GoalTemplate),
    !,
    format('details (~w, ~w ~w)~n', [Category, BrandToFind,
ModelToFind]),
    print_all_attributes(1, Arity, Category, GoalTemplate).
```

```

print_all_attributes(N, Arity, _, _) :-
    N > Arity,
    nl.
print_all_attributes(N, Arity, Category, ProductTerm) :-
    N <= Arity,
    ( N == 1 ->
        true
    ;
        arg(N, ProductTerm, Value),
        ( attribute_index(Category, AttrNameAtom, N) ->
            AttrName = AttrNameAtom
        ;
            atom_concat('Attribute ', N, AttrName)
        ),
        format('  ~w - ~w~n', [AttrName, Value])
    ),
    N1 is N + 1,
    print_all_attributes(N1, Arity, Category, ProductTerm).

```

Листинг 3.5 Правило для поиска товара по бренду и модели

Примечание Источник: собственная разработка

Этот предикат инициирует поиск, вызывая вспомогательный предикат `find_product_in_categories/4`, который рекурсивно перебирает список `Categories`. Для каждой категории `find_product_in_categories/4` пытается найти и вывести детали с помощью `try_find_and_print_details/3`. Предикат `try_find_and_print_details/3` для заданной категории формирует шаблон `GoalTemplate`, подставляя в него искомые `BrandToFind` и `ModelToFind` в соответствующие позиции (второй и третий аргумент факта). Затем выполняется `call(GoalTemplate)`. Если товар найден, используется отсечение `(!/0)` для предотвращения поиска дальнейших (возможно, дублирующихся) записей и выводится заголовок с категорией, брендом и моделью. После этого вызывается `print_all_attributes/4` для печати всех атрибутов найденного товара.

Вспомогательный рекурсивный предикат `print_all_attributes/4` отвечает за последовательный вывод атрибутов. Он принимает текущий номер атрибута `N`, общую арность `Arity`, категорию и терм найденного продукта. Атрибут с номером 1 (`ProductID`) пропускается. Для остальных атрибутов извлекается их значение `Value` и имя `AttrName` через `attribute_index/3`, после чего они выводятся в форматированном виде. Процесс повторяется до тех пор, пока не будут выведены все атрибуты.

Создание меню программы

Для обеспечения удобного взаимодействия между пользователем и справочно-информационной системой был добавлен модуль program.pl. Он отвечает за отображение меню опций, считывание выбора пользователя и вызов соответствующих предикатов из rules.pl для выполнения запрошенных операций. Взаимодействие осуществляется через консольный интерфейс, точкой входа в программу является предикат `start_program/0`, который инициирует основной цикл.

Предикат `start_menu` реализует основной цикл взаимодействия с пользователем. Он использует встроенную продукцию `repeat/0`, которая создает бесконечный цикл, прерываемый только явным указанием на завершение. В каждой итерации цикла последовательно вызываются `show_menu/0` для отображения доступных опций, `read_option/1` для считывания выбора пользователя, и `process_option/1` для обработки выбранной опции. Цикл завершается, когда `process_option/1` обрабатывает опцию выхода (например опция 0) и вызывает «`!, fail`», что приводит к прерыванию `repeat/0`.

Полный код файла program.pl находится в приложении А.

Предикат `show_menu/0` отвечает за вывод на консоль списка доступных опций. Он использует стандартные предикаты вывода `nl/0` (новая строка, «New Line») и `write/1` для форматированного отображения текстового меню, включающего опции для сортировки товаров, сравнения двух товаров, фильтрацию по критериям, поиск детальной информации о товаре, а также опцию выхода из программы. Это обеспечивает интуитивно понятное представление функционала системы.

Для получения ответа от пользователя имеется предикат `read_option(Option)`. Сначала с помощью `read_line_to_string(user_input, Line)` считывается вся введенная пользователем строка. Это позволяет обрабатывать и корректный, и ошибочный ввод. Затем делается попытка преобразовать полученную строку `Line` в числовое значение `Option` с использованием `atom_number(Line, Option)`. Для достижения отказоустойчивости данная операция «обернута» в конструкцию `catch/3`. В случае, если преобразовать строку в число невозможно (например, при вводе текста), `catch/3` перехватывает ошибку, выводит сообщение «Invalid input. ...» и присваивает переменной `Option` специальное значение `-1`, сигнализирующее об ошибке ввода, что позволяет корректно обработать ситуацию в предикате.

Логика выбора действия на основе введенной пользователем строки (или команды) определена в предикате `process_option(Option)`:

- При Option равном 0: выводится сообщение «Exiting program.» и программа успешно завершается.
- При Option равном 1: вызывается предикат `handle_sort_products/0`, который организует процесс сортировки товаров (`find_sorted_by_attribute/2`).

Методы `handle_compare_products/0`, `handle_filter_products/0` и `handle_find_details/0` вызываются при Option равном 2, 3 и 4 соответственно. Все они вызывают необходимые предикаты из файла `rules.pl`.

Продукция `handle_sort_products/0` организует диалог с пользователем для получения необходимых данных для сортировки. С помощью `write/1` он выводит запросы на ввод категории товаров (например, «gpr» или «monitor») и атрибута, по которому будет проводиться сортировка (например, «price» или «brand»). Ввод пользователя считывается как атом с помощью вспомогательного предиката `read_line_to_atom/2`. После получения обоих атомов (`CategoryAtom` и `AttributeAtom`) и базовой проверки их атомарности, управление переходит предикату `find_sorted_by_attribute(CategoryAtom, AttributeAtom)` из модуля `rules.pl`, который уже непосредственно выполняет поиск, сортирует товары по выбранной категории и выводит данные. В случае некорректного формата ввода выводится соответствующее сообщение.

Для выполнения операции сравнения двух товаров предназначены предикат `handle_compare_products/0`. Он поочередно запрашивает у пользователя: категорию сравниваемых товаров (`CategoryAtom`), а затем бренд и модель для первого товара (`Brand1Str`, `Model1Str`), бренд и модель второго товара (`Brand2Str`, `Model2Str`). Ввод для категории считывается как атом, а для брендов и моделей — как строки, которые затем преобразуются в термы с помощью `term_string/2`. После сбора всех пяти параметров и проверки их атомарности (для категории и преобразованных брендов/моделей) вызывается предикат `compare_products(CategoryAtom, Brand1, Model1, Brand2, Model2)` из `rules.pl` для отображения таблицы характеристик. При неверном формате ввода пользователю выводится сообщение.

Предикат `handle_filter_products/0` обеспечивает интерфейс для наиболее гибкой функции системы — фильтрации товаров по набору критериев. Сначала у пользователя запрашивается категория товаров (`CategoryAtom`). Затем выводится приглашение ввести список критериев в виде стандартного Prolog-терма, например, `[price =< 500, brand == 'AMD']`. Для корректного считывания этого структурированного списка используется встроенный предикат `read_term(CriteriaList, [variable_names(VNames)])`, который позволяет парсить сложные термы. После проверки, что категория является атомом, а критерии действительно представляют собой список, вызывается

предикат `find_products(CategoryAtom, CriteriaList)` из модуля `rules.pl` для выполнения фильтрации и вывода результатов.

Для поиска и отображения полной информации о конкретном товаре служит предикат `handle_find_details/0`. Он запрашивает у пользователя бренд (`BrandStr`) и модель (`ModelStr`) искомого продукта. Введенные пользователем строки преобразуются в атомарные термы `Brand` и `Model` с помощью `term_string/2`. После успешного преобразования и проверки на атомарность, эти данные передаются предикату `find_product_details(Brand, Model)` из модуля `rules.pl`, который осуществляет поиск товара по всем категориям и выводит его детальные характеристики.

3. Выводы

В данной главе был подробно описан процесс проектирования и создания справочно-информационной системы «Покупка товаров для гейминга» на языке Prolog. Основой системы стала структурированная база знаний, реализованная в виде набора фактов, которая описывает восемь категорий товаров для гейминга и их ключевые характеристики. Проектирование фактов было нацелено на компактное и логичное представление данных.

Для обеспечения функциональности системы был разработан комплекс предикатов (правил) Prolog. Эти правила реализуют основные операции: сортировку товаров по выбранной характеристике, наглядное сравнение двух товаров одной категории, гибкую фильтрацию по множественным критериям с использованием различных операторов, а также вывод полной информации о конкретном товаре. Универсальность ключевых предикатов была достигнута за счет работы с индексами атрибутов и динамического формирования целей.

Взаимодействие с пользователем организовано через текстовый консольный интерфейс, реализованный в модуле `program.pl`. Меню программы построено на циклическом отображении опций, считывании выбора и диспетчеризации вызовов к основным правилам обработки данных. Модульная структура программы, включающая файлы `facts.pl`, `rules.pl` и `program.pl`, способствует ее пониманию, поддержке и потенциальному дальнейшему усовершенствованию. Таким образом, в результате выполнения описанных этапов была создана работоспособная система, демонстрирующая применение языка Prolog для решения задач построения справочно-информационных систем.

ГЛАВА 4. ТЕСТИРОВАНИЕ И ТРАССИРОВКА ПРОГРАММЫ

1. Анализ работоспособности программы

После проектирования и создания справочно-информационной системы ее необходимо протестировать. Тестирование будет проходить в два этапа — от лица пользователя и с использованием трассировки для более детального анализа работы программы.

Сперва будет выполнен тест запуска программы через команду «start_menu.» и выход из программы при помощи нулевого пункта в меню.

```
?- cd('/Users/daniel/Documents/Prolog').
true.

?- make.
true.

% facts.pl compiled 0.00 sec, 56 clauses
% rules.pl compiled 0.01 sec, 113 clauses
Warning: /Users/daniel/Documents/Prolog/program.pl:79:
Warning: Singleton variables: [VNames]
% /Users/daniel/Documents/Prolog/program.pl compiled 0.01 sec, 16 clauses
Welcome! Write "start_menu." to begin.
?- start_menu.

Gaming Products Information Menu
1. Sort products by attribute
2. Compare two products
3. Filter products by criteria
4. Find product details by brand and model
0. Exit
Choose an option: 0

Exiting program.
false.

?-
```

Рисунок 4.1 – Демонстрация запуска и выхода из программы

Примечание Источник: собственная разработка

Как видно из рисунка, система справляется с задачами запуска программного цикла и выходом.

Теперь стоит проверить работу каждого каждого правила.

```
Gaming Products Information Menu
1. Sort products by attribute
2. Compare two products
3. Filter products by criteria
4. Find product details by brand and model
0. Exit
Choose an option: 1

Enter category (gpu, mouse, ...): headset
Enter attribute to sort by (price, brand, ...): low_hz

headset sorted by "low_hz"
Razer BlackShark V2 X - low_hz: 12
HyperX Cloud Alpha - low_hz: 13
HyperX Cloud 2 - low_hz: 15
SteelSeries Arctic 7+ - low_hz: 20
Redragon Diomedes - low_hz: 20
SteelSeries Arctic Nova 5 - low_hz: 20
Redragon Zeus X - low_hz: 50

true ;
Invalid option. Please try again.
true ;

Gaming Products Information Menu
1. Sort products by attribute
2. Compare two products
3. Filter products by criteria
4. Find product details by brand and model
0. Exit
Choose an option:
```

Рисунок 4.2 – Демонстрация работы правила find_sorted_by_attribute/2

Примечание Источник: собственная разработка

На фото можно наблюдать список гарнитур, отсортированных по нижней границе диапазона частот. Программа корректно выводит список гарнитур, что хорошо видно на фото.

Также на фото можно наблюдать повторный вызов меню после выполнения предиката.

Проверим все остальные предикаты на правильную работу.

```
Gaming Products Information Menu
1. Sort products by attribute
2. Compare two products
3. Filter products by criteria
4. Find product details by brand and model
0. Exit
Choose an option: 2

Enter category (cpu, gpu, ...): gpu
Enter brand of first product: 'Afox'
Enter model of first product: 'GeForce GTX 1050 Ti'
Enter brand of second product: 'Palit'
Enter model of second product: 'GeForce RTX 3050 StormX'

gpu: Afox GeForce GTX 1050 Ti <vs> Palit GeForce RTX 3050 StormX
brand: Afox <vs> Palit
model: GeForce GTX 1050 Ti <vs> GeForce RTX 3050 StormX
price: 507 <vs> 722
gpu_chip_brand: NVIDIA <vs> NVIDIA
vram_gb: 4 <vs> 6
vram_type: GDDR5 <vs> GDDR6
interface: PCI Express x16 3.0 <vs> PCI Express x8 4.0
tech_process_nm: 14 <vs> 8
base_clock_mhz: 1290 <vs> 1040
boost_clock_mhz: 1392 <vs> 1470
threads: 768 <vs> 2304

true
```

Рисунок 4.3 – Демонстрация работы предиката compare_products/5

Примечание Источник: собственная разработка

```
Gaming Products Information Menu
1. Sort products by attribute
2. Compare two products
3. Filter products by criteria
4. Find product details by brand and model
0. Exit
Choose an option: 3

Enter category (monitor, ram, ...): monitor
Enter criteria list ([price =< 500, brand == 'AMD', ...]): [diagonal_inch > 25, price =< 800].

found products
Xiaomi Gaming Monitor G27i
Asus TUF Gaming
Philips Evnia

true .
```

Рисунок 4.4 – Выполнение продукции find_products/2

Примечание Источник: собственная разработка

```
Gaming Products Information Menu
1. Sort products by attribute
2. Compare two products
3. Filter products by criteria
4. Find product details by brand and model
0. Exit
Choose an option: 4

Enter brand of the product: 'Kingston'
Enter model of the product: 'NV3'

details (storage, Kingston NV3)
brand - Kingston
model - NV3
price - 199
storage_type - SSD
capacity_gb - 1000
interface - PCI Express x4 4.0
form_factor_inch - M.2
read_speed_mbps - 6000
write_speed_mbps - 4000
rpm - none

true
```

Рисунок 4.5 – Выполнение продукции find_product_details/2

Примечание Источник: собственная разработка

Таким образом, при тестировании программы проблем или сбоев не было обнаружено. От лица пользователя, все предикаты выдают верный результат и работают корректно.

2. Анализ с использованием трассировки

Для более глубокого понимания механизма работы разработанной справочно-информационной системы и проверки корректности логики выполнения предикатов, помимо тестирования от лица пользователя, целесообразно провести анализ с использованием трассировки. Трассировка в Prolog представляет собой пошаговое отслеживание процесса вывода, которое позволяет увидеть, как система пытается удовлетворить

поставленную цель (запрос), какие правила и факты используются, как происходит унификация переменных и как осуществляется механизм возврата (бэктрэкинга) при поиске решений. Использование трассировки является мощным инструментом отладки и верификации логических программ, так как оно наглядно демонстрирует весь путь вычислений и помогает выявить потенциальные ошибки или неэффективные участки кода.

В рамках данного анализа будет рассмотрена трассировка выполнения одного из ключевых предикатов системы — `find_products/2`, отвечающего за фильтрацию товаров по заданным критериям. Это позволит детально изучить процесс сопоставления критериев с фактами из базы знаний и убедиться в правильности работы логики фильтрации.

Будем проводить трассировку следующего запроса к базе знаний: `find_products(monitor, [diagonal_inch > 25, price =< 800])`.

Для детального отслеживания работы механизма фильтрации были установлены `trace`-точки на следующие предикаты с помощью команды `trace`:

- `trace(find_products/2)`. для отслеживания основного вызова предиката фильтрации.
- `trace(check_criteria/3)`. для наблюдения за рекурсивной проверкой каждого критерия из списка для каждого товара.

Эти точки были выбраны таким образом, чтобы увидеть основные этапы работы алгоритма фильтрации, не перегружая вывод трассировки излишними деталями.

Выполним запрос с трассировкой.

```

?- find_products(monitor, [diagonal_inch > 25, price =< 800]).
T [12] Call: find_products(monitor, [diagonal_inch>25, price=<800])
T [24] Call: check_criteria([diagonal_inch>25, price=<800], monitor(monitor1,
'Xiaomi', 'Gaming Monitor G27i', 523, 27, '1920x1080', 165, 'IPS', 'AMD FreeSync
Premium', '16:9', 'HDR10'), monitor)
T [31] Call: check_criteria([price=<800], monitor(monitor1, 'Xiaomi', 'Gaming Monitor
G27i', 523, 27, '1920x1080', 165, 'IPS', 'AMD FreeSync Premium', '16:9', 'HDR10'),
monitor)
T [38] Call: check_criteria([], monitor(monitor1, 'Xiaomi', 'Gaming Monitor G27i', 523,
27, '1920x1080', 165, 'IPS', 'AMD FreeSync Premium', '16:9', 'HDR10'), monitor)
T [38 +0.2ms] Exit: check_criteria([], monitor(monitor1, 'Xiaomi', 'Gaming Monitor
G27i', 523, 27, '1920x1080', 165, 'IPS', 'AMD FreeSync Premium', '16:9', 'HDR10'),
monitor)
T [31 +0.6ms] Exit: check_criteria([price=<800], monitor(monitor1, 'Xiaomi', 'Gaming
Monitor G27i', 523, 27, '1920x1080', 165, 'IPS', 'AMD FreeSync Premium', '16:9',
'HDR10'), monitor)
T [24 +1.1ms] Exit: check_criteria([diagonal_inch>25, price=<800],
monitor(monitor1, 'Xiaomi', 'Gaming Monitor G27i', 523, 27, '1920x1080', 165, 'IPS',
'AMD FreeSync Premium', '16:9', 'HDR10'), monitor)
T [24] Call: check_criteria([diagonal_inch>25, price=<800], monitor(monitor2, 'LG',
'Ultra Gear 24GS60F-B', 556, 23.8, '1920x1080', 180, 'IPS', 'AMD FreeSync', '16:9',
'HDR10'), monitor)
T [24 +0.2ms] Fail: check_criteria([diagonal_inch>25, price=<800], monitor(monitor2,
'LG', 'Ultra Gear 24GS60F-B', 556, 23.8, '1920x1080', 180, 'IPS', 'AMD FreeSync',
'16:9', 'HDR10'), monitor)
T [24] Call: check_criteria([diagonal_inch>25, price=<800], monitor(monitor3, 'Asus',
'Rog Strix XG27ACS', 1199, 27, '2560x1440', 180, 'IPS', 'AMD FreeSync', '16:9',
'HDR10'), monitor)

```

Рисунок 4.6 – Результаты трассировки предиката `find_products/2`

Примечание Источник: собственная разработка

Представленный фрагмент трассировки иллюстрирует работу системы при обработке запроса на фильтрацию мониторов. Далее проведем детальный анализ результатов трассировки.

- Строка 1 (T [12] Call: ...) показывает начальный вызов основного предиката фильтрации `find_products/2` с указанной категорией `monitor` и списком критериев `[diagonal_inch > 25, price =< 800]`. Предикат `find_products/2` внутри себя использует `findall/3` для перебора всех фактов категории `monitor`. Для каждого найденного факта (представляющего конкретный монитор) вызывается предикат `check_criteria/3` для проверки соответствия всем заданным критериям.
- Строка 2 (T [24] Call: ...) демонстрирует первый вызов `check_criteria/3` для первого монитора из базы знаний. Предикат получает полный список критериев.

- Строка 3 (T [31] Call: ...) рекурсивный вызов `check_criteria/3` для того же `monitor1`, но уже с оставшимся списком критериев [`price =< 800`].
- Строка 4 и 5 (T [38] Call: ... и T [38 +0.2ms] Exit: ...) достигнут базовый случай рекурсии для `check_criteria/3` — список критериев пуст. Это означает, что `monitor1` удовлетворяет всем условиям. Предикат успешно завершается (Exit).
- Строка 6 и 7 (T [31 +0.6ms] Exit: ... и T [24 +1.1ms] Exit: ...) последовательные успешные завершения (Exit) для рекурсивных вызовов `check_criteria/3` для `monitor1`.
- Строка 8 (T [24] Call: ...) предикат `findall/3` (внутри `find_products/2`) переходит к следующему факту — `monitor2`. `check_criteria/3` снова вызывается с полным списком критериев.
- Строка 9 (T [24 +0.2ms] Fail: ...) вызов `check_criteria/3` для `monitor2` завершается неудачей (Fail), так как он не удовлетворяет первому критерию (диагональ). Это значит, что `monitor2` не будет включен в итоговый список отфильтрованных товаров.
- И так далее.

Представленный фрагмент трассировки наглядно демонстрирует логику работы предикатов `find_products/2` и `check_criteria/3`. Видно, как система последовательно перебирает товары указанной категории и для каждого из них рекурсивно проверяет соответствие всем заданным критериям. В случае `monitor1`, оба критерия (диагональ > 25 и цена ≤ 800) были удовлетворены, что привело к успешному завершению всех вложенных вызовов `check_criteria/3`. Для `monitor2` первый же критерий (диагональ > 25) не был выполнен, что корректно привело к неудаче (Fail) предиката `check_criteria/3` для данного товара, и он не был включен в результирующий набор.

Трассировка подтверждает, что механизм фильтрации работает в соответствии с заложенной логикой, корректно обрабатывая как удовлетворяющие, так и не удовлетворяющие критериям случаи.

Для полного анализа можно было бы включить в трассировку предикаты `attribute_index/3`, чтобы увидеть процесс извлечения значений и их непосредственного сопоставления.

3. Выводы

В данной главе было проведено комплексное тестирование разработанной справочно-информационной системы «Покупка товаров для гейминга». Тестирование от лица пользователя включало проверку всех

основных функций: запуск и выход из программы, сортировку товаров по атрибуту, сравнение характеристик двух товаров одной категории, фильтрацию товаров по разным критериям, а также поиск детальной информации о конкретном товаре. Результаты тестирования показали, что система корректно обрабатывает пользовательские запросы и выдает ожидаемые результаты в соответствии с заложенной логикой и данными из базы знаний. Проблем или сбоев в работе основных функций при корректном вводе обнаружено не было.

Для более глубокого анализа работы системы был применен механизм трассировки, встроенный в SWI-Prolog. На примере выполнения предиката `find_products/2` с критериями `find_products(monitor, [diagonal_inch > 25, price <= 800])` была продемонстрирована пошаговая работа логики фильтрации. Установка точек трассировки на предикаты `find_products/2` и `check_criteria/3` позволила наглядно увидеть, как система перебирает факты из базы знаний (мониторы) и рекурсивно проверяет соответствие каждого товара заданным критериям (`diagonal_inch > 25` и `price <= 800`). Трассировка подтвердила корректность механизма сопоставления и отсеивания товаров, не удовлетворяющих условиям, а также успешное прохождение всех проверок для товаров, соответствующих запросу.

Таким образом, проведенное тестирование и анализ с использованием трассировки подтвердили работоспособность и корректность функционирования разработанной справочно-информационной системы на языке Prolog. Система успешно справляется с поставленными задачами по предоставлению информации о товарах для гейминга, их сортировке, сравнению и фильтрации. Трассировка позволила убедиться в правильности реализации логического вывода и обработки данных.

ГЛАВА 5. ПОСТРОЕНИЕ ДЕРЕВЬЕВ ЦЕЛИ С ИСПОЛЬЗОВАНИЕМ ЯЗЫКА UML

1. Построение UML-дерева

Для наглядной демонстрации процесса логического вывода, применяемого разработанной справочно-информационной системой, было решено построить дерево цели для одного из ключевых правил. В качестве такого запроса был выбран предикат `find_products/2`, отвечающий за фильтрацию товаров, так как он включает рекурсивную проверку множественных критериев и хорошо иллюстрирует механизм работы Prolog. Для анализа был взят конкретный вызов: `find_products(monitor, [diagonal_inch > 25, price =< 800])`. Предварительный анализ выполнения этого запроса с помощью механизма трассировки позволил выявить основные шаги логического вывода, которые легли в основу построения дерева.

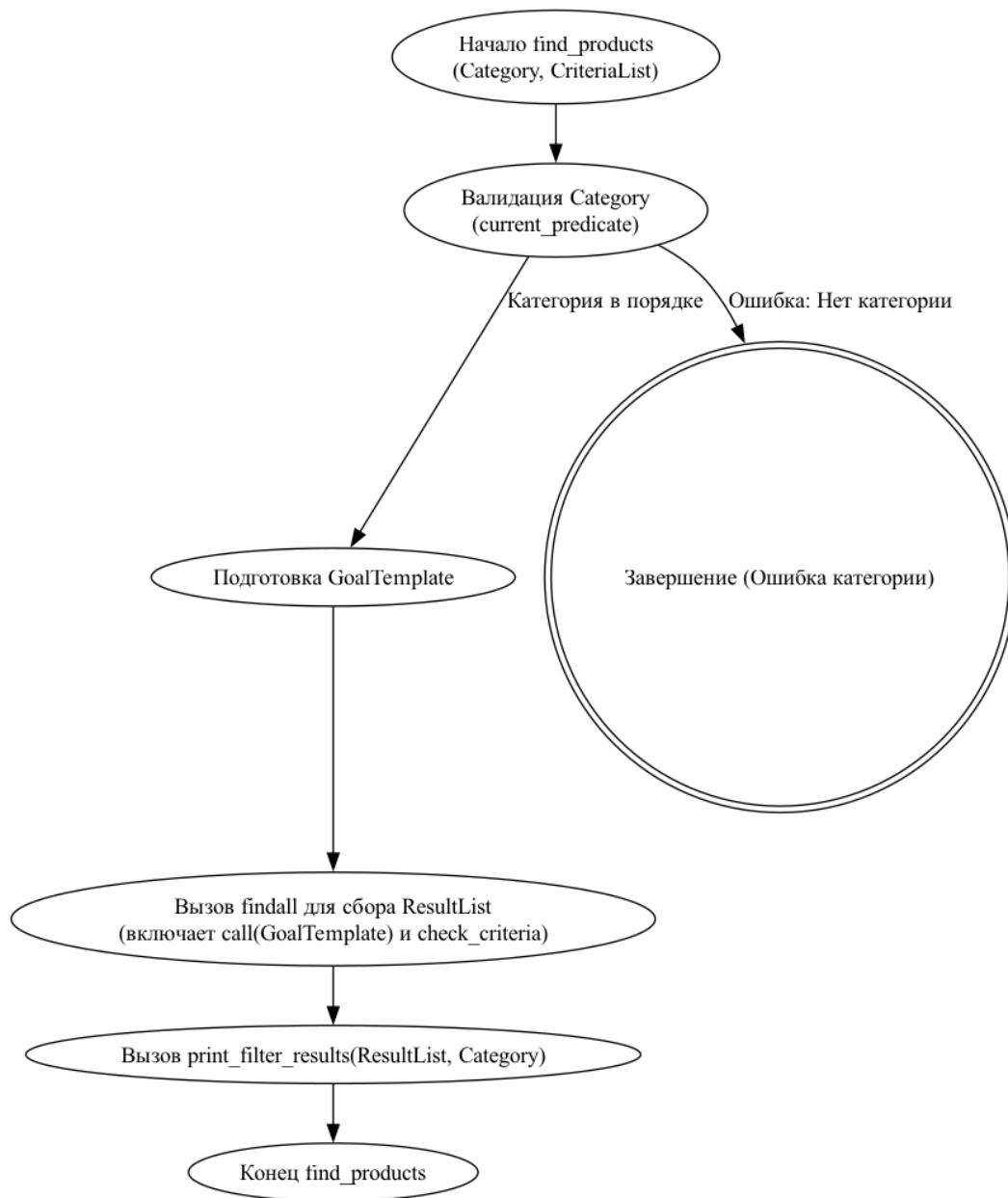


Рисунок 5.1 – Дерево предиката find_products/2

Примечание Источник: собственная разработка

На рисунке представлена обобщенная схема работы (дерево цели, визуализированное как диаграмма UML) предиката find_products/2. Данная диаграмма иллюстрирует основные логические шаги, которые выполняет система при обработке запроса на фильтрацию товаров по заданным критериям. Она не показывает обработку каждого конкретного факта из базы знаний, а отражает общую последовательность действий и точки принятия решений внутри предиката.

Работа предиката начинается с получения категории Category и списка критериев CriteriaList. Первым шагом является «Валидация Category». В случае ошибки работа завершается. При успехе система переходит к «Подготовке GoalTemplate» для последующего перебора фактов.

Центральным этапом является «Вызов findall для сбора ResultList». Предикат findall/3 интегрирует по всем товарам указанной категории, и для каждого товара вызывает предикат check_criteria/3. Этот вспомогательный предикат рекурсивно проверяет соответствие текущего товара всем критериям из CriteriaList. Товары, прошедшие все проверки, добавляются в ResultList.

После формирования списка ResultList происходит «Вызов print_filter_result(ResultList, Category)», который отвечает за отображение найденных товаров. Если ResultList пуст, выводится соответствующее уведомление, иначе печатается список подходящих товаров. Затем работа предиката find_products/2 завершается.

2. Выводы

В данной главе была рассмотрена методика визуализации процесса логического вывода в системе Prolog с использованием деревьев цели, представленных в нотации UML. Основное внимание было уделено построению дерева цели для ключевого предиката find_products/2, отвечающего за фильтрацию товаров.

Использование UML-диаграмм для визуализации дерева цели способствует лучшему пониманию структуры предиката и механизма работы логического вывода в Prolog. Это также может служить полезным инструментом для анализа и отладки сложных логических программ. Построенная диаграмма для find_products/2 подтвердила корректность заложенной в предикат логики и продемонстрировала его способность обрабатывать составные запросы на фильтрацию данных.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данного курсового проекта была успешно решена задача проектирования, создания и обработки на языке Prolog справочно-информационной системы «Покупка товаров для гейминга». Работа охватила как теоретические аспекты логического программирования, так и практическую реализацию функциональной системы.

На начальном этапе были изучены фундаментальные принципы декларативной парадигмы программирования и ключевые концепции языка Prolog, включая его историю, основные элементы (факты, правила, запросы, термы) и механизмы работы, такие как унификация и бэктрекинг. Было проведено математическое обоснование структуры и синтаксиса Prolog с точки зрения исчисления предикатов первого порядка, что позволило глубже понять формальные основы языка.

Центральной частью работы стал анализ предметной области «Товары для гейминга». Были определены восемь основных категорий товаров и их ключевые атрибуты, важные для пользователя при выборе. На основе этого анализа была спроектирована и реализована база знаний в виде набора фактов Prolog, компактно и структурированно представляющая информацию о 56 конкретных товарах.

Для обеспечения взаимодействия с базой знаний и выполнения пользовательских запросов был разработан комплекс предикатов (правил). Реализованы функции поиска детальной информации о товаре по бренду и модели, сортировки товаров внутри категории по различным атрибутам, сравнения характеристик двух товаров одной категории, а также универсальная функция фильтрации товаров по списку критериев с использованием различных операторов сравнения. Разработка этих правил продемонстрировала гибкость Prolog в обработке информации и реализации сложной логики.

Для взаимодействия пользователя с системой был создан текстовый консольный интерфейс. Модульная структура программы, разделенная на файлы для фактов, правил и логики интерфейса, обеспечивает ее лучшую читаемость, поддержку и потенциал для дальнейшего расширения.

Проведенное тестирование системы, включавшее как проверку функциональности от лица пользователя, так и анализ с использованием механизма трассировки SWI-Prolog для предиката `find_products/2`, подтвердило корректность работы всех заявленных функций. Система адекватно обрабатывает запросы, предоставляет точные результаты и демонстрирует правильность реализации логического вывода. Построение дерева цели с использованием нотации UML для ключевого предиката

фильтрации позволило наглядно представить сложный процесс принятия решений системой.

Таким образом, цель курсового проекта была достигнута: создана работоспособная справочно-информационная система на языке Prolog. Выполненная работа позволила получить практический опыт в разработке систем с использованием декларативной парадигмы и логического языка программирования. Возможными улучшениями проекта могут стать расширение базы знаний, реализация более сложных критериев поиска, разработка графического пользовательского интерфейса и интеграция с внешними источниками данных для актуализации информации о товарах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Логическое программирование в SWI-prolog : учеб. пособие / Е. М. Товбис, С. П. Якимов ; СибГУ им. М. Ф. Решетнева. — Красноярск, 2022. — 86 с.
2. Логическое программирование : учебное пособие / Ю.В. Новицкая. — Новосибирск: Изд-во НГТУ, 2021. — 86 с.
3. Интернет-ресурс Wikipedia [Электронный ресурс] / Интернет-портал. – Режим доступа: https://ru.wikipedia.org/wiki/Декларативное_программирование. - Дата доступа: 21.05.25.
4. Зюзьков В. М. Математическое введение в декларативное программирование: учебное пособие. — Томск: ТГУ, 2003. — 83 с.

ПРИЛОЖЕНИЕ А

```
:- consult('rules.pl').
```

```
start_menu :-
```

```
    repeat,  
        show_menu,  
        read_option(Option),  
        process_option(Option),  
        ( Option == 0 ->  
            !, fail  
        ;  
        ).
```

```
show_menu :-
```

```
    nl,  
    write('Gaming Products Information Menu'), nl,  
    write('1. Sort products by attribute'), nl,  
    write('2. Compare two products'), nl,  
    write('3. Filter products by criteria'), nl,  
    write('4. Find product details by brand and  
model'), nl,  
    write('0. Exit'), nl,  
    write('Choose an option: ').
```

```
read_option(Option) :-
```

```
    read_line_to_string(user_input, Line),  
    catch(atom_number(Line, Option), _,  
    (write('Invalid input. Please enter a number.'), nl,  
    Option = -1)),  
    nl.
```

```
process_option(0) :-
```

```
    write('Exiting program.'), nl.
```

```
process_option(1) :-
```

```
    handle_sort_products.
```

```
process_option(2) :-
```

```
    handle_compare_products.
```

```

process_option(3) :-
    handle_filter_products.
process_option(4) :-
    handle_find_details.
process_option(_) :-
    write('Invalid option. Please try again. '), nl.

handle_sort_products :-
    write('Enter category (gpu, mouse, ...): '),
    read_line_to_atom(user_input, CategoryAtom),
    write('Enter attribute to sort by (price,
brand, ...): '),
    read_line_to_atom(user_input, AttributeAtom),
    nl,
    ( atom(CategoryAtom), atom(AttributeAtom) ->
        find_sorted_by_attribute(CategoryAtom,
AttributeAtom)
    ;
        write('Invalid category or attribute
format. '), nl
    ).

handle_compare_products :-
    write('Enter category (cpu, gpu, ...): '),
    read_line_to_atom(user_input, CategoryAtom),
    write('Enter brand of first product: '),
    read_line_to_string(user_input, Brand1Str),
    term_string(Brand1, Brand1Str),
    write('Enter model of first product: '),
    read_line_to_string(user_input, Model1Str),
    term_string(Model1, Model1Str),
    write('Enter brand of second product: '),
    read_line_to_string(user_input, Brand2Str),
    term_string(Brand2, Brand2Str),
    write('Enter model of second product: '),
    read_line_to_string(user_input, Model2Str),
    term_string(Model2, Model2Str),
    nl,
    ( atom(CategoryAtom), atom(Brand1), atom(Model1),
atom(Brand2), atom(Model2) ->

```

```

        compare_products(CategoryAtom, Brand1, Model1,
Brand2, Model2)
    ;
    write('Invalid input format for comparison. '),
nl
    ).

handle_filter_products :-
    write('Enter category (monitor, ram, ...): '),
    read_line_to_atom(user_input, CategoryAtom),
    write('Enter criteria list ([price =< 500, brand
== \'AMD\', ...]): '),
    read_term(CriteriaList, [variable_names(VNames)]),
nl,
    ( atom(CategoryAtom), is_list(CriteriaList) ->
        find_products(CategoryAtom, CriteriaList)
    ;
        write('Invalid category or criteria list
format. '), nl
    ).

handle_find_details :-
    write('Enter brand of the product: '),
    read_line_to_string(user_input, BrandStr),
term_string(Brand, BrandStr),
    write('Enter model of the product: '),
    read_line_to_string(user_input, ModelStr),
term_string(Model, ModelStr),
nl,
    ( atom(Brand), atom(Model) ->
        find_product_details(Brand, Model)
    ;
        write('Invalid brand or model format. '), nl
    ).

read_line_to_atom(Stream, Atom) :-
    read_line_to_string(Stream, String),
    atom_string(Atom, String).

:- initialization( (write('Welcome! Write
"start_menu." to begin. '), nl) ).

```